UMBC Accessibility GPS

System Design Document (SDD)

Team Member Names: Aidan Denham, Alex Marbut, Celestine Sumah, Ethan Michalik

1. Introduction....3
1.1. Purpose of This Document....3
1.2. References....3
2. System Architecture....3
2.1. Architectural Design....3
2.2. Decomposition Description....5
3. Persistent Data Design....6
3.1. Database Descriptions....6
3.2. File Descriptions....7
4. Requirements Matrix....9
Appendix A - Agreement Between Customer and Contractor....10
Appendix B - Peer Review Sign-off....11
Appendix C - Document Contributions....11

1. Introduction

1.1. Purpose of This Document

1.1.1. The purpose of this document is to outline the system design for the UMBC Accessibility GPS website. It will outline the systems architecture and how it functions in order to satisfy project requirements. The intended audience of this document is the product owner, the stakeholder (who is also the customer), and the developers to ensure we are meeting the needs of the project.

1.2. References

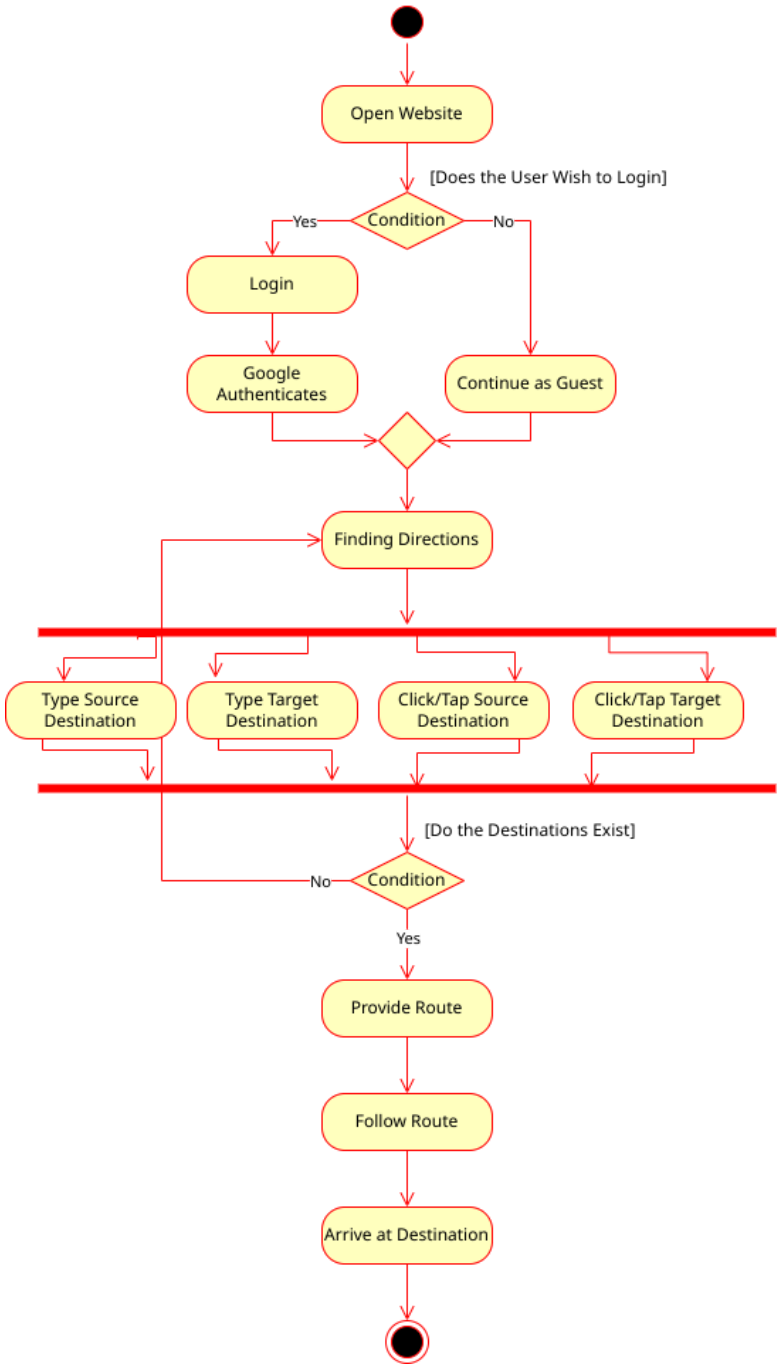
1.2.1. SRS Document - UMBC Accessibility GPS

1.2.2. UMBC Accessibility Routes Map

2. System Architecture

2.1. Architectural Design

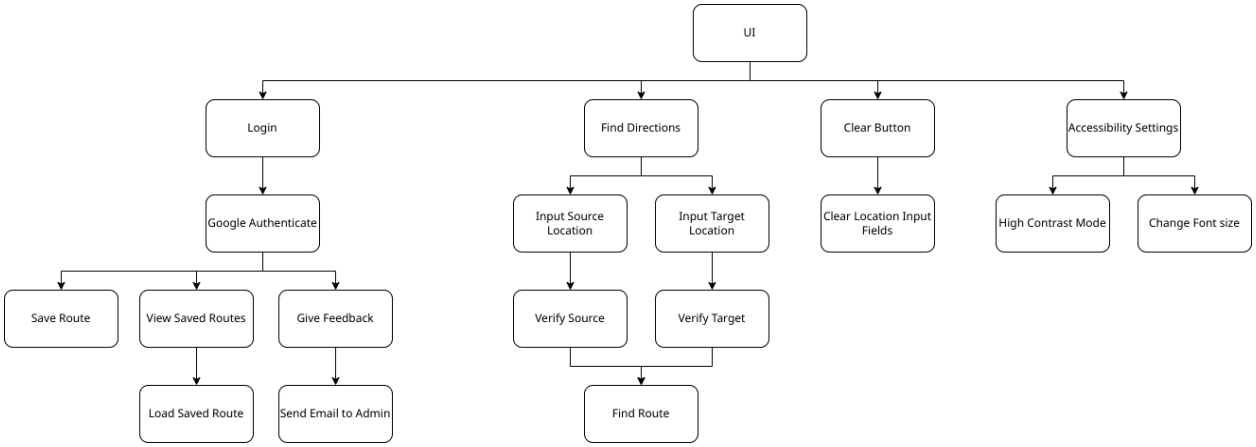
2.1.1. Our front end is built using HTML, CSS, JavaScript, and TypeScript, utilizing Leaflet.js to create our interactable map. On the backend, data related to pathing is stored in a GeoJSON file and called by the program. The code for our website is hosted on our github, available at https://github.com/pairOfPants/umbc-gps.



2.1.2. Above is an activity diagram of our system. First the user will open the website and choose to either login to their UMBC account or continue as a guest. Next, the user will be able to find directions by typing in their source destination and target destination. The system will then verify both these destinations exist (i.e. Not ITE Room 999). If they do not exist, the user will have to change the destinations they gave. If they both exist, the system will calculate the route between their destinations and provide them to the user. The user will then follow the route given until they reach

their destination. Once they reach their destination, the app will tell them they have made it to their destination.

2.2. Decomposition Description



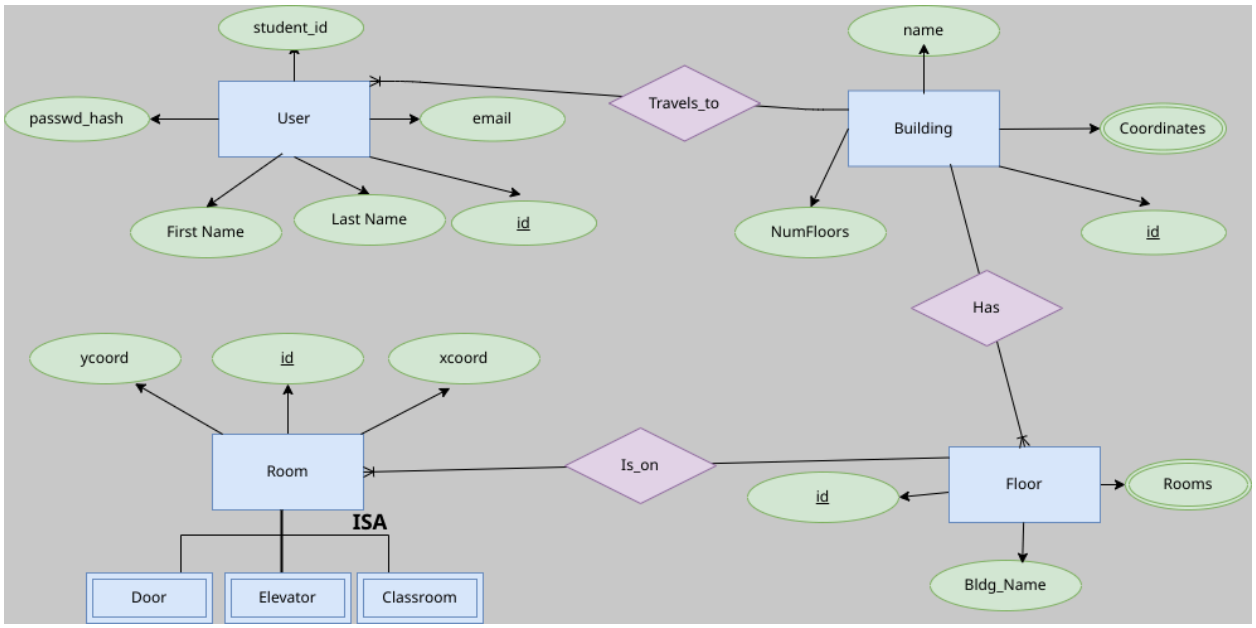
2.2.1. Our system uses a functional design. Starting from the left of our diagram, the login function enables users to sign in with Google, which the system verifies to ensure that they are using a UMBC email. Once authenticated, the user will have access to features such as saving and loading routes, viewing saved routes, and giving feedback about the website.

Moving to functions related to finding directions, the user will be able to input a source destination, whether that be their current location or somewhere else on campus. The user will also input a target destination for where they want to go. The system will then verify both these places exist. Then the system will provide the user with a route from their source destination to their target destination. The users will also have access to a clear button, which clears the current destination input fields.

The user will also have access to extra accessibility settings for their liking. They can choose between normal, or high contrast mode. They can also choose to increase font size for the website.

3. Persistent Data Design

3.1. Database Descriptions



3.1.1. User

- 3.1.1.1. student_id: string - Student's UMBC ID
- 3.1.1.2. email: string - Student's UMBC email
- 3.1.1.3. passwd_hash: string - Student's secure password hash
- 3.1.1.4. first_name: string - Student's first name
- 3.1.1.5. last_name: string - Student's last name
- 3.1.1.6. id: string - Entity's ID in the database

3.1.2. Building

- 3.1.2.1. name: string - Name of the building
- 3.1.2.2. coordinates: geographical point (GeoJSON) - geographic coordinates of the building
- 3.1.2.3. num_floors: int - Number of floors in the building
- 3.1.2.4. id: string - Entity's ID in the database

3.1.3. Floor

- 3.1.3.1. bldg_name: string - Name of the building the floor is in
- 3.1.3.2. rooms: string array - List of all rooms available on the floor
- 3.1.3.3. id: string - Entity's ID in the database

3.1.4. Room

- 3.1.4.1. ycoord: float (GeoJSON) - Global y-coordinate of the room
- 3.1.4.2. xcoord: float (GeoJSON) - Global x-coordinate of the room
- 3.1.4.3. id: string - Entity's ID in the database

3.2. File Descriptions

3.2.1. Our website only uses one external file. This file is our GeoJSON file, which stores all of the path data on campus for our map. This file was originally downloaded from OpenStreetMap and then tweaked by us to add additional routes, thus the format will be the same to any other GeoJSON file downloaded from OpenStreetMap. Here is a small example snippet of the formatting:


```
{
    "type": "FeatureCollection",
    "generator": "JOSM",
    "features": [
        {
            "type": "Feature",
            "properties": {
                "tiger:name_type": "Pl",
                "name": "Circle Place",
                "tiger:cfcc": "A41",
                "tiger:name_base": "Circle",
                "tiger:county": "Baltimore, MD",
                "tiger:zip_right": "21227",
                "tiger:reviewed": "position",
                "highway": "residential",
                "tiger:zip_left": "21227"
            },
            "geometry": {
                "type": "LineString",
                "coordinates": [
                    [
                        -76.70391260000,
                        39.25870910000
                    ],
                    [
                        -76.70406280000,
                        39.25832900000
                    ],
                    [
                        -76.70420700000,
                        39.25793950000
                    ]
                ]
```

```
            }
        },
        {
            "type": "Feature",
            "properties": {
                "highway": "footway"
            },
            "geometry": {
                "type": "LineString",
                "coordinates": [
                    [
                        -76.71321420000,
                        39.25385970000
                    ],
                    [
                        -76.71317600000,
                        39.25393960000
                    ]
                ]
            }
        }
......... etc.
```

4. Requirements Matrix

4.1. Below is our requirements matrix, showing which system components satisfy each of the functional requirements from the SRS.

Functional Requirement	System Components	Location
Use Case 1: Find Directions Outdoors	Input Source Destination	UI (Top bar of main page)
	Input Target Destination	UI (Top bar of main page)
	Select Clear Button	UI (Top bar of main page)
	Clear Input Fields	clearInput()
	Verify Destinations	verifyLocations()
	Find Route	calculateRoute()
Use Case 2: User Logs In	Select Log In Button	UI (Start page, and top of main page)
	Login Through Google	googleLogin()
	Log Out	googleLogout()
Use Case 3: Get Directions Indoors	Input Source Destination	UI (Top bar of main page)
	Input Target Destination	UI (Top bar of main page)
	Select Clear Button	UI (Top bar of main page)
	Clear Input Fields	clearInput()
	Verify Destinations	verifyLocations()
	Find Route	calculateRoute()
Use Case 4: User Provides Feedback	Select Feedback Button	UI (Bottom of main page)
	Input feedback	UI (Feedback Page)
	Submit feedback	sendFeedbackEmail()
Use Case 5: Admin Manages User Feedback	N/A	Done through Admin's email.
Use Case 6: Admin Updates Website	All	Update source code up to standards

Appendix A - Agreement Between Customer and Contractor

By signing this document, both the customer and the developers agree to the details specified in this document. Specifically, the scope of the project, the functional requirements listed, and the non-functional requirements listed. By signing this document the customer also agrees to cooperate with the development team for any future questions they may have.

If the customer wishes to make any changes to this document, they must first talk to the development team to make those changes. Then, both parties must agree to those changes and re-sign this document. Then the version number will be updated along with all of the changes documented.

Customer

Name	Signature	Date	Comments

Team

Name	Signature	Date	Comments
Aidan Denham	Aidan Denham	11/16/2025	
Celestine Sumah	Celestine Sumah	11/16/2025	n/a
Ethan Michalik	Ethan Michalik	11/16/2025	
Alex Marbut	Alex Marbut	11/16/2025	

Appendix B - Peer Review Sign-off

By signing this document, all members of the development team have reviewed this document and agreed to its contents and format. If there are any major points of contention, please refrain from signing until they have been addressed.

Name	Signature	Date	Comments
Celestine Sumah	Celestine Sumah	11/16/2025	n/a
Aidan Denham	Aidan Denham	11/16/2025	
Ethan Michalik	Ethan Michalik	11/16/2025	
Alex Marbut	Alex Marbut	11/16/2025	

Appendix C - Document Contributions

Contributions by: Aidan Denham, Alex Marbut, Celestine Sumah, Ethan Michalik